ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF COMPUTER AND
INFORMATICS

TinyNPU

Graduation Project Final Report

Fırat Kızılboğa
150200115

Department: Computer Engineering
Division: Computer Engineering

Advisor: Assoc. Prof. Dr. Ayse Yilmazer

March 2026

Statement of Authenticity

I/we hereby declare that in this study

1. all the content influenced from external references are cited clearly and in detail,
2. and all the remaining sections, especially the theoretical studies and implemented software/hardware that constitute the fundamental essence of this study is originated by my/our individual authenticity.

İstanbul, Mart 2026

Fırat Kızılboğa

Acknowledgments

I would like to thank my advisor, Assoc. Prof. Dr. Ayse Yilmazer, for her guidance throughout this project. I also thank the people whose open-source tooling and research made it possible to build, analyze, and validate the TinyNPU hardware and software stack.

TinyNPU

(SUMMARY)

TinyNPU is a compact accelerator implemented as a complete hardware/software stack. It combines a parameterized systolic-array neural processing unit with a PyTorch-first segmented compiler and runtime, a quantization-aware model-preparation flow, RTL and host-side verification infrastructure, and performance and synthesis characterization tooling. The stack is designed to make the datapath usable from a realistic machine-learning workflow and measurable in a disciplined way.

The hardware architecture is organized around an $N \times N$ output-stationary systolic array, a unified buffer, an instruction memory, streamer logic that feeds the array, and a post-processing unit that applies bias, rescaling, activation, and packed writeback. The software stack traces PyTorch models, partitions them into accelerator segments and host operations, compiles supported segments into TinyNPU instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation. Unsupported work is never hidden; it appears explicitly as host-side steps in the execution plan.

The useful unit of design is the whole stack, not the RTL alone. Compiler-visible tensor semantics, physical packing format, host/NPU boundaries, verification points, and repeated-run runtime behavior all affect whether the accelerator is practical to use. TinyNPU therefore includes explicit host-op dispatch, compiler-owned expected tensors for verification, repeated-run sessions with persistent state, and benchmark accounting that separates accelerator compute cycles from runtime and protocol overhead.

The implemented stack has been validated end-to-end on a trained MNIST model, and a controlled multi-tile matrix multiplication benchmark is used to study compute speedup and integration overhead. In addition, a reproducible Yosys-based synthesis flow provides rough resource estimates for the current design. The report presents TinyNPU as a working accelerator research system for studying architecture, compilation, packing, and workload fit together.

TinyNPU

(ÖZET)

TinyNPU, sistolik dizi tabanlı bir nöral işlemci birimini onu kullanılabilir hale getiren derleyici, çalışma zamanı, niceleme akışı ve benzetim tabanlı doğrulama ortamıyla birlikte ele alan bütünleşik bir donanım-yazılım sistemidir. Amaç yalnızca küçük bir hızlandırıcı çekirdeği tasarlamak değil, bu çekirdeği gerçek bir makine öğrenmesi iş akışında kullanılabilir hale getirmektir. Bu nedenle sistem, PyTorch tabanlı model hazırlama sürecini destekleyen, nicelemeli modelleri hızlandırıcı segmentlerine dönüştüren, desteklenmeyen işlemleri ana işlemci üzerinde çalıştıran ve sonuçları yazılım referanslarıyla doğrulayan bir yapı olarak tasarlanmıştır.

Donanım tarafında TinyNPU, birleşik tampon bellek ve komut belleği ile beslenen sistolik dizi tabanlı bir hesaplama çekirdeği etrafında kurulmuştur. Girdi aktivasyonları ve ağırlıklar birleşik tampondan akıtıcı birimlere, oradan da sistolik diziye iletilir. Hesaplama sonucunda oluşan değerler son işleme biriminden geçirilerek paketlenmiş biçimde tekrar belleğe yazılır. Bu yapı, yoğun nicelemeli matris işlemleri için verimli bir veri yolu sağlarken, dışarıdan görülen programlama modelini de yönetilebilir düzeyde tutmaktadır. Yazılım tarafı ise yüksek seviyeli model işlemlerini hızlandırıcı segmentleri ve gerekirse ana işlemci işlemlerini birlikte içeren bir yürütme planına dönüştürmektedir.

Bu çalışmada sistem, eğitilmiş MNIST modeli dahil uçtan uca iş yükleri üzerinde doğrulanmıştır. Ayrıca, hızlandırıcının gerçek hesaplama çevrimlerini paketleme, arayüz ve çalışma zamanı maliyetlerinden ayıran bir ölçüm altyapısı kurulmuştur. Böylece çekirdeğin kendi verimliliği ile entegrasyon maliyetleri ayrı ayrı değerlendirilebilmektedir. Tekrarlı çıkarım senaryoları için oturum tabanlı çalışma ve ağırlık kalıcılığı desteği de eklenmiştir.

Rapor, bu sistemi donanım mimarisi, model hazırlama ve derleme akışı, doğrulama yöntemi ve deneysel sonuçlarıyla birlikte sunmaktadır. Sonuçlar, sistolik çekirdeğin güçlü hesaplama sağladığını, ancak toplam davranışın hâlâ veri yerleşimi, bölütleme ve çalışma zamanı maliyetlerine duyarlı olduğunu göstermektedir.

Contents

1 Introduction and Project Summary **1**
1.1 Background and Motivation 1
1.2 Problem Definition 1
1.3 Project Scope and Approach 1

2 Comparative Literature Survey **3**
2.1 Mixed-Precision Quantization 3
2.2 High-Performance Matrix Multiplication 3
2.3 Existing Accelerator Architectures 3

3 Developed Approach and System Model **5**
3.1 System Overview 5
3.2 NPU 5
3.2.1 Unified Buffer and Physical Roles 6
3.2.2 Systolic Array and Skewers 7
3.2.3 Processing Element 7
3.2.4 Post-Processing Unit 8
3.2.5 Control and Instruction Format 9
3.3 Quantizer and Model Preparation 12
3.4 Compiler 14
3.5 Runtime and Verification 16
3.6 Current Scope Boundaries 18

4 Experimental Setup and Design **19**

5 Comparative Evaluation and Discussion **20**
5.1 Kernel-Level Benchmark 20
5.2 Repeated-Run Execution-Plan Case Study 21
5.3 Activation Study 23
5.4 Bottlenecks and Interpretation 23

6 Conclusion and Future Work **25**

A Implemented Activation Algorithms **25**

B Benchmark Cost Model **27**

1 Introduction and Project Summary

1.1 Background and Motivation

Modern deep neural networks demand substantial compute and memory bandwidth even at inference time. This has made specialized accelerator design a central topic both for datacenter-scale systems and for smaller edge-oriented deployments [8, 4]. Quantization is one of the main techniques that makes such deployment practical, because it reduces memory traffic and allows arithmetic throughput to increase when narrow integer types are used instead of floating-point data.

However, quantization is not a single fixed target. Mixed-precision work has shown that different layers or kernels inside the same model can tolerate different bit-widths [11]. This creates a hardware opportunity and a software problem at the same time. Hardware should benefit from narrower arithmetic when it is safe to do so, but the software stack must still prepare the model, preserve numerical intent, and decide where accelerator execution ends and host execution begins.

1.2 Problem Definition

The main problem addressed by TinyNPU is therefore not only how to build a small systolic-array accelerator, but how to make mixed-precision quantized computation usable from a realistic model flow. Fixed-precision accelerators leave throughput on the table when different parts of a model have different precision needs. At the other end of the spectrum, CPU-centric approaches based on ISA extensions or soft SIMD can support low-bit computation, but they still pay substantial software-side cost for packing, unpacking, and orchestrating the data movement around those operations [3, 1, 10].

This project treats the problem as a co-design task. The hardware must expose a regular datapath that benefits from packed low-precision arithmetic. The compiler and runtime must make tensor layout, quantization boundaries, and host fallbacks explicit rather than implicit. The result should be a system that can be measured honestly: accelerator compute, runtime overhead, and host-side work should not be conflated.

1.3 Project Scope and Approach

TinyNPU approaches this problem with four coupled components:

1. **A parameterized mixed-precision accelerator:** TinyNPU is centered around an output-stationary $N \times N$ systolic array, a unified buffer, streamer logic that feeds the array, and a post-processing unit that performs bias addition, rescaling, activation, saturation, and packed writeback.

2. **Packed integer computation:** The processing elements support packed INT16, INT8, and INT4 execution modes. Throughput therefore scales with precision reduction, not by changing the global architecture, but by packing more useful work into the same datapath.

3. **A segmented compiler/runtime:** The software flow traces supported PyTorch models, partitions them into accelerator segments and explicit host-side operations, lowers legal accelerator work into instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation.

4. **Verification and measurement infrastructure:** The project includes compiler-owned expected tensors, runtime verification points, repeated-run sessions, and benchmark accounting that separates accelerator compute from protocol and software overhead.

TinyNPU runs a trained MNIST model through the full compiler/runtime stack and benchmarks a controlled multi-tile matrix multiplication workload. These experiments expose the architecture, packing behavior, compiler boundaries, and runtime cost of the current system.

The rest of this report follows that logic. The next section positions TinyNPU relative to relevant literature in mixed-precision quantization, matrix multiplication, and accelerator design. The system model section then explains the implemented hardware and software stack in detail, followed by the experimental method, the current results, and the most important next steps.

2 Comparative Literature Survey

2.1 Mixed-Precision Quantization

Mixed-precision quantization work is the clearest algorithmic motivation for TinyNPU. Wu et al. formulate layer-wise quantization as a search problem and show that different layers can be assigned different bit-widths without treating the whole model uniformly [11]. This is the key observation behind a variable-precision accelerator: if model layers do not all need the same precision, a fixed-precision datapath cannot exploit the full efficiency opportunity.

Quantization is also relevant as a compiler concern rather than only a hardware concern. SmoothQuant is an example of pushing difficult numerical adjustments into an offline transformation so that runtime execution becomes simpler and more hardware-friendly [12]. TinyNPU follows the same general philosophy even though it does not implement the full SmoothQuant method itself: model preparation, fused rescaling, and compiler-ready conversion are treated as software responsibilities that simplify the accelerator contract.

2.2 High-Performance Matrix Multiplication

The mathematical core of TinyNPU is still matrix-style computation, so the project is strongly informed by classical GEMM literature. Goto and van de Geijn show that high-performance matrix multiplication depends not only on arithmetic throughput, but on careful tiling and data movement through the memory hierarchy [5]. The same principle appears in TinyNPU at a smaller scale. When matrices exceed the dimensions of the systolic array, the compiler and runtime must tile along the M, N, and K dimensions while preserving the data reuse pattern expected by the array and its post-processing path.

2.3 Existing Accelerator Architectures

On the accelerator side, Google's TPU remains the canonical reference for systolic-array-style matrix acceleration [8]. Gemmini is especially relevant because it demonstrates how a systolic array, programming model, and system context can be treated as one full-stack research object rather than as an isolated RTL block [4]. That full-stack perspective is one of the strongest methodological reference points for TinyNPU.

An earlier educational TinyTPU implementation was useful as a starting point for understanding systolic-array organization, but it was not a good endpoint for this project [2]. Extending it toward mixed precision, tiled execution, and cleaner post-processing exposed the limits of treating the datapath as the whole problem. TinyNPU therefore diverged into a custom implementation with a different software contract, explicit host/accelerator boundaries, and a compiler/runtime that owns packing and

verification rather than leaving those concerns informal.

Taken together, these references define the design space in which TinyNPU sits. It is not trying to outscale large industrial accelerators. Its niche is a compact mixed-precision accelerator system in which the hardware, packing model, compiler/runtime, and measurement methodology can all be studied together in one working platform.

3 Developed Approach and System Model

The system is described here as four cooperating subsystems: the TinyNPU hardware itself, the quantizer and model-preparation path, the compiler, and the runtime. This organization matches the real engineering boundaries of the stack and makes the movement of tensors, instructions, and packed data explicit.

3.1 System Overview

At a high level, the flow is straightforward. A model starts either as a quantized PyTorch module or as a compiler-ready artifact produced by the quantization path. The compiler traces the model, partitions it into accelerator-supported regions and explicit host operations, lowers legal regions into TinyNPU instruction and buffer images, and packages the result as a compiled artifact. The runtime then executes that artifact step by step, loading the accelerator when needed, invoking host-side operations when needed, and checking expected tensors when verification is enabled. Inside the hardware, the unified buffer, streamers, systolic array, and post-processing unit implement the actual matrix-style execution path.

That end-to-end view is important because the main design problem is not only to build a datapath. It is to maintain a clean contract between logical tensors, physical packed layouts, host-side fallback behavior, and the actual hardware pipeline. The rest of this section therefore explains the system in the same order: hardware first, then model preparation, then compilation, then runtime execution and verification.

3.2 NPU

The TinyNPU hardware is organized around a control path and a compute path. The control path is responsible for MMIO-driven loading, instruction sequencing, and control-state progression. The compute path consists of the unified buffer, streamer logic, a parameterized $N \times N$ output-stationary systolic array, and the post-processing unit. During active computation, data is read from the unified buffer, oriented by the streamers, consumed by the processing elements, and then drained through the post-processing unit before being written back in packed form.

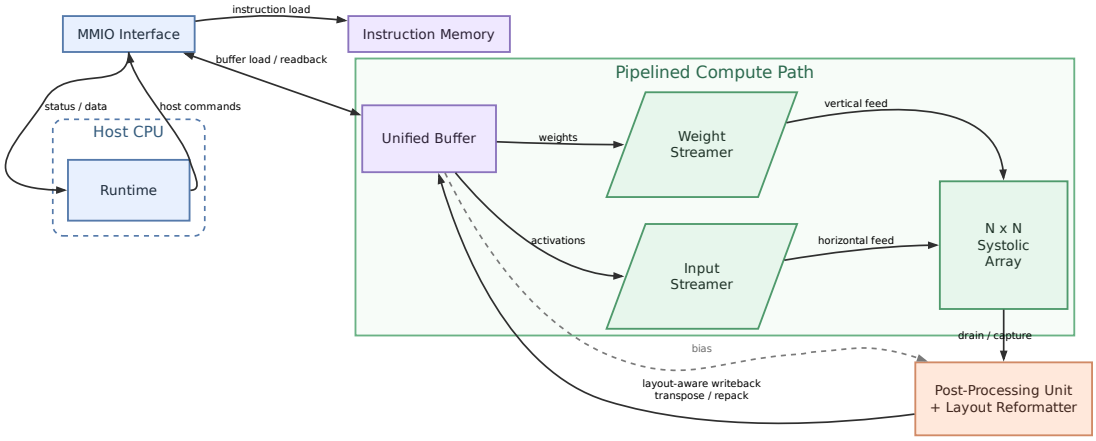

Figure 3.1: Top-level TinyNPU hardware architecture.

Figure 3.1 shows the top-level organization used throughout this report. The host-side runtime communicates with the accelerator through the MMIO interface, which is responsible for instruction loading and buffer access. Inside the accelerator, the unified buffer, input streamer, weight streamer, and systolic array form the main pipelined compute path. The post-processing unit is shown separately because it becomes active during the later drain and writeback phase rather than during the main feed phase of the array. This distinction is important for explaining both data movement and measured execution cost.

3.2.1 Unified Buffer and Physical Roles

The unified buffer is not just a generic scratchpad in this design. Architecturally, it behaves as a true wide dual-port memory with a masked write path. One read side serves the activation stream, while the other serves the weight stream; occasional control-unit random reads are arbitrated onto the activation-side port. This form was chosen to keep both array feed directions active at the same time. A narrower or less concurrent memory interface would turn buffer access into the bottleneck and leave the systolic array underutilized. Because the buffer is wide and row-oriented, this organization is also compatible with a banked memory implementation rather than requiring a single monolithic custom memory block.

Within that buffer, the legacy names A, B, C, and BIAS refer to physical roles, not to immutable tensor semantics. A is the left-fed operand: its tiles are packed so that each unified-buffer row can launch one horizontal activation vector into the systolic array boundary. B is the top-fed operand: its tiles are packed for the vertical weight stream and the corresponding skewer delays. C is the writeback region: it stores quantized output tiles in the packed format chosen by the output precision, so its physical layout may differ from the logical matrix view. BIAS is a separate region consumed only by

the PPU, which loads one output-column bias vector before draining a tile. These names therefore describe how data is consumed by the hardware pipeline, not whether a tensor was logically an activation, a weight, or an intermediate result in the original model. In the current RTL, this distinction is enforced more explicitly than in the earlier prototype: an internal output is no longer assumed to be universally reusable. The compiler now tags each NPU-produced tensor with the physical layout required by its next consumer, and the writeback path can emit either ordinary C-style output packing or an A-compatible transposed/repacked form for chained matmuls.

3.2.2 Systolic Array and Skewers

The array is output-stationary in the literal RTL sense. Packed activation words enter from the left edge and move one PE to the right per cycle. Packed weight words enter from the top edge and move one PE downward per cycle. What remains stationary is the output accumulator inside each PE: over the full reduction along the K dimension, every PE repeatedly adds its local partial sum into the same 64-bit register. Only after the last K contribution has been consumed does the array switch into drain mode and shift those accumulated results downward into the post-processing unit. This choice fits the dominant tiled-GEMM workload well because it keeps the reduction local to the PE and avoids repeatedly moving partial sums through the buffer system during the K loop.

The streamer/skewer logic is what makes this wavefront legal. It delays row i by $i + 1$ cycles so that the activation and weight packets that belong to the same dot-product term meet on the correct PE at the correct cycle. A tile therefore executes in three phases: feed the packed K-steps into the array, allow the wavefront to fill across the remaining diagonals, and then drain the stationary accumulators.

The skewer sits between the unified buffer and the systolic array and adds small row- and column-dependent delays to the incoming streams. On the activation side, row i is delayed by i extra cycles before it enters the left edge; on the weight side, column j is delayed by j extra cycles before it enters the top edge. The values themselves do not change. What changes is when they enter the array. That timing offset creates the diagonal wavefront expected by the systolic schedule, so the activation and weight pieces that belong together meet at the correct PE in the correct cycle. Without the skewer, the data would enter as a flat burst and the alignment inside the array would be wrong.

3.2.3 Processing Element

The processing element is where mixed precision becomes concrete. Each PE latches one 16-bit packed activation word and the lower 16 bits of the vertical bus as the current packed weight word. In INT16 mode that word holds one signed multiply; in INT8 mode it holds two signed byte multiplies whose products are summed; in INT4 mode it holds four signed nibble multiplies whose products are summed. The resulting 32-bit partial sum is sign-extended and accumulated into a 64-bit stationary register.

One subtle but important implementation detail is that the vertical PE-to-PE path is 64 bits wide even though the compute datapath consumes only 16-bit packed weights. This is not wasted width. During compute, only `data_from_top[15:0]` is interpreted as the incoming packed weight word and the full 64-bit bus is simply delayed and passed downward. During drain, the same physical column bus is re-used to shift full 64-bit accumulators from one row to the next. The shared bus keeps the PE interface uniform across compute and drain instead of requiring a second dedicated drain network.

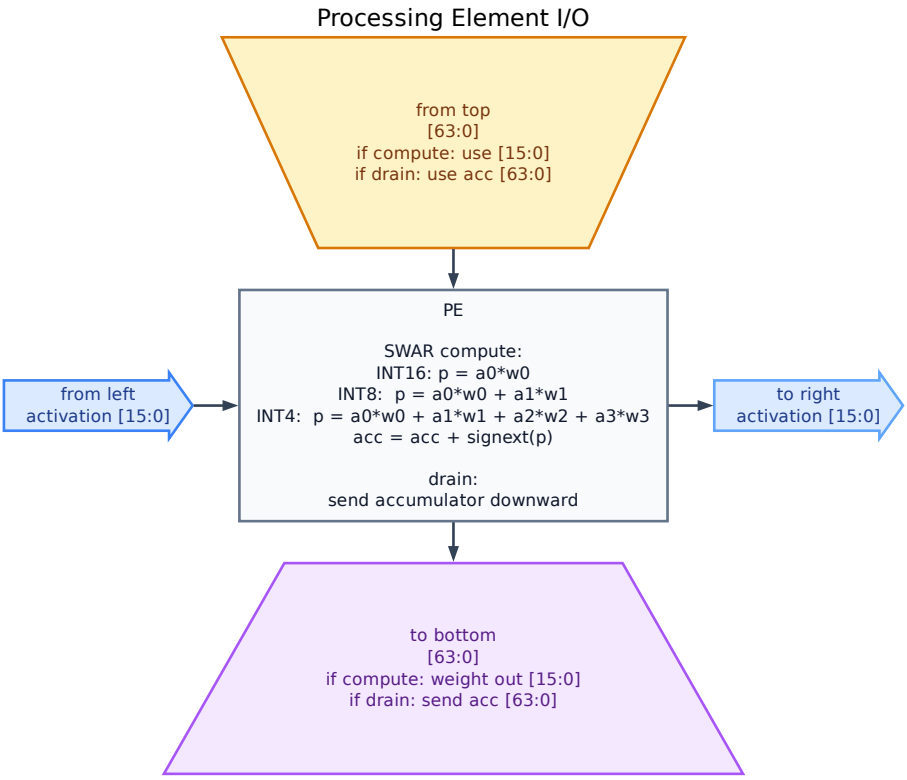

Figure 3.2: Processing-element I/O in compute and drain modes.

3.2.4 Post-Processing Unit

The post-processing unit (PPU) is best understood as an N-lane row processor backed by an internal $N \times N$ tile buffer. During drain, it receives one N-element accumulator row vector per cycle from the bottom of the systolic array, processes all N lanes of that row in parallel, and stores the quantized result into its internal tile buffer. Only after the full tile has been captured does it begin writing buffered rows back to the unified buffer. The current PPU therefore is not a second streaming mesh; it is a row-wise post-processing stage followed by buffered writeback.

In the implemented RTL, the PPU first loads the bias vector in two chunks, four 32-bit bias values at a time. For each drained row it then evaluates the same requantization

pipeline lane by lane:

$$q = \mathrm{clip}_p\left(\mathrm{act}\left(\left((acc + bias) \times multiplier + 2^{shift-1}\right) \gg shift\right)\right)$$

where *acc* is the 64-bit drained accumulator, *bias* is a sign-extended 32-bit bias term, *multiplier* is the 16-bit fixed-point requantization factor, $\gg$ is an arithmetic right shift, $\mathrm{act}(\cdot)$ selects the configured post-processing nonlinearity, and $\mathrm{clip}_p(\cdot)$ saturates to the signed range of the selected output precision. In the current stack, the activation stage supports plain ReLU, an integer sigmoid path derived from a DI-Exp-style integer nonlinear primitive in the spirit of I-LLM [7], and a hardware-friendly hard-GELU approximation used when GELU is lowered into the accelerator. The GELU side is motivated by the original GELU formulation and its cheaper approximate forms [6], together with I-BERT's observation that simpler hard-GELU-style operators are cheaper than fuller integer GELU constructions but also less accurate [9]. TinyNPU therefore chose the simpler hard-GELU path for the current PPU rather than a more accurate I-GELU-style implementation, because the latter would have added substantially more hardware complexity to the post-processing stage. In RTL terms, this is implemented as bias addition, 65-by-17-bit multiplication, round-to-nearest by adding $2^{shift-1}$ before the shift when $shift > 0$, the selected activation, and final saturation to INT16, INT8, or INT4.

Writeback is also precision-dependent. After the tile buffer has been filled, the PPU emits one buffered row per writeback cycle. For INT16 output, each 16-bit lane is written back directly. For INT8 and INT4 output, the PPU shifts the quantized value into the byte or nibble slot selected by the packed write offset, and the unified buffer applies a bit mask so that only that subword is updated while previously written sibling tiles in the same physical word are preserved. The control unit derives this packed write offset from the output tile index: two logical M-tiles share one physical word for INT8, and four logical M-tiles share one physical word for INT4. A newer writeback-layout selector extends this path beyond the earlier "C is good enough" assumption. When the compiler marks an internal output as the next layer's A-side input, the PPU reformats its buffered tile during writeback into an A-compatible packed layout. For INT16 this behaves like a small tile transpose; for INT8 and INT4 it becomes a transpose-plus-subword repack that places the byte or nibble groups where the left-fed streamer expects them. This layout-aware writeback fixed a real chained-matmul correctness bug in the project and is now part of the validated hardware contract.

3.2.5 Control and Instruction Format

This behavior is programmed through a fixed-width 256-bit instruction format in instruction memory. The current `MATMUL` instruction encodes the opcode, base addresses for the A, B, C, and bias regions in the unified buffer, the tile counts along M, K, and N, and the PPU configuration fields for multiplier, shift, activation, input precision, output precision, hard-GELU metadata, and output-layout selection. The control unit therefore does not fetch a stream of tiny scalar operations; it fetches a coarse-grained instruction that describes one tiled matrix-multiplication region and then steps through clear, feed, wait, drain, bias-load, and writeback states internally. This ISA was kept

deliberately simple because the supported workload is highly predictable. A finer-grained instruction stream would add fetch and decode overhead without providing much useful flexibility in the current compiler contract.

Table 3.1: Implemented `MATMUL` instruction field layout in the current RTL. The decode path now consumes both hard-GELU metadata and output-layout selection for chained writeback. Remaining low bits are presently unused.

Bits	Field	Meaning
[255 : 252]	`opcode`	Operation code; `ISA_OP_MATMUL` selects the tiled matrix-multiplication sequencer.
[247 : 232]	`A_base`	Unified-buffer base address of A tiles in activation layout.
[231 : 216]	`B_base`	Unified-buffer base address of B tiles in weight layout.
[215 : 200]	`C_base`	Unified-buffer base address of C writeback tiles.
[183 : 168]	`M_tiles`	Number of output-tile positions along the M dimension.
[167 : 152]	`K_tiles`	Number of reduction passes per output tile.
[151 : 136]	`N_tiles`	Number of output-tile positions along the N dimension.
[135 : 120]	`BIAS_base`	Unified-buffer base address of the bias payload consumed by the PPU.
[119 : 112]	`shift`	Arithmetic right-shift amount for requantization.
[111 : 96]	`multiplier`	Fixed-point requantization multiplier applied in the PPU.
[95 : 88]	`activation`	Post-processing activation selector consumed by the PPU; current software uses it for ReLU, integer sigmoid, and hard-GELU modes.
[87 : 86]	`out_precision`	Output packing precision used by the PPU writeback path.
[85 : 84]	`write_offset`	Reserved packed-lane field in the current instruction format. In the present RTL, the effective write offset is derived internally from output-tile index and output precision during packed writeback.
[83 : 82]	`in_precision`	Input precision mode broadcast to the systolic array PEs.
[81 : 74]	`h_gelu_x_scale_shift`	Per-op hard-GELU input-domain shift used by the PPU when GELU is lowered into the accelerator.
[73 : 72]	`output_layout`	Physical writeback layout selector. The current RTL supports ordinary C-style output packing plus an A-compatible transposed/repacked form for chained matmuls.

The RTL decode path also recognizes a small set of simpler opcodes around this main instruction. `HALT` terminates the current instruction stream and places the control unit in the halted state, `NOP` advances without side effects, and `MOVE` is reserved for internal data movement. In the current JIT flow, `MATMUL` is the important emitted

operation and `HALT` marks the end of the segment program.

Table 3.2: Implemented instruction opcodes in the current RTL control path.

Opcode	Name	Role
`0x0`	`NOP`	No operation; decode advances without performing accelerator work.
`0x1`	`HALT`	Terminates execution and leaves the controller in the halted state until the host issues another command.
`0x2`	`MATMUL`	Coarse-grained tiled matrix-multiplication instruction with integrated PPU configuration.
`0x3`	`MOVE`	Internal data-movement operation reserved by the RTL control path.

Instruction execution is paired with a small MMIO command interface on the host side. The host does not directly drive internal control states; instead it writes a command register to request memory access or to start execution from a given instruction-memory address.

Table 3.3: Host-visible MMIO commands used to service the accelerator.

Command	Value	Meaning
`CMD_WRITE_MEM`	`0x01`	Writes the current MMIO data value into unified-buffer or instruction-memory space at the selected address.
`CMD_READ_MEM`	`0x02`	Reads the selected memory location back into the MMIO-visible data register.
`CMD_RUN`	`0x03`	Starts instruction execution from the instruction-memory address carried in the argument register.

The control path is also observable, not only functional. The controller exposes internal state counters when performance monitoring is enabled, which makes it possible to see how much time is spent in fetch, decode, feed, wait, drain, and writeback states instead of treating the whole accelerator run as a black box.

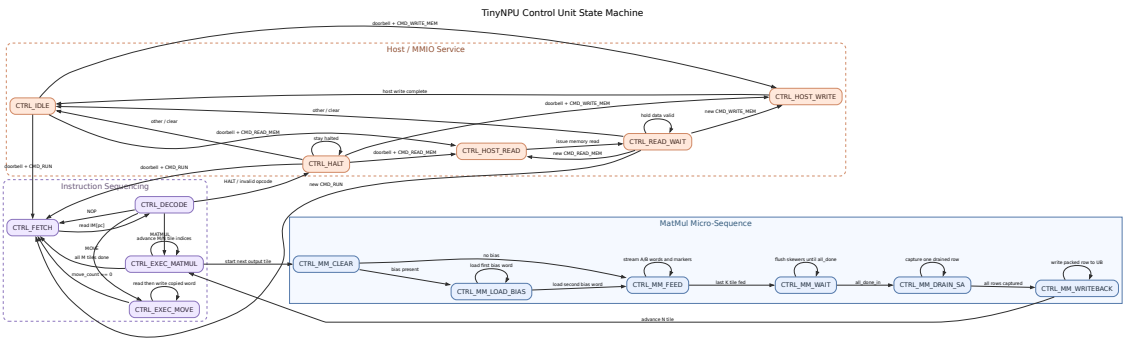

Figure 3.3: Control-unit states tracked by the internal performance counters, grouped by host service, instruction sequencing, and the matrix-multiplication micro-sequence.

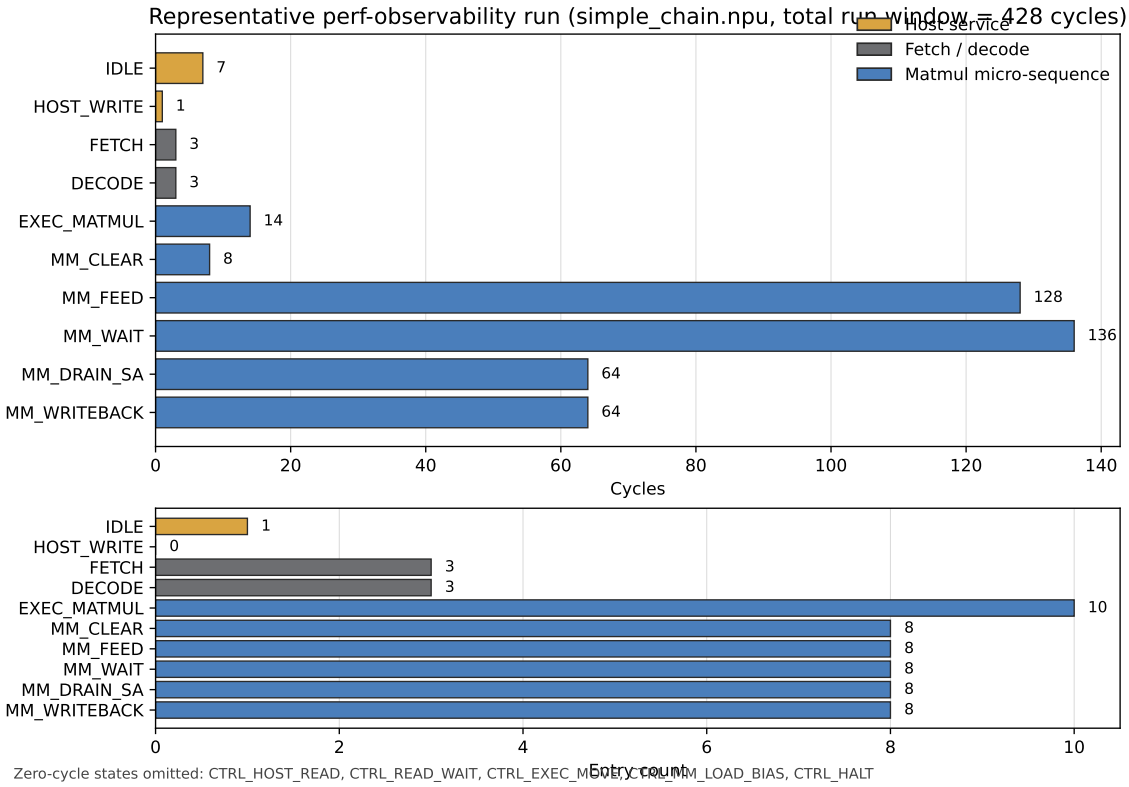

Figure 3.4: Measured per-state cycle and entry breakdown for a representative perf-observability run on `simple_chain.npu`. Zero-cycle states are omitted for readability.

The measured breakdown in Figure 3.4 makes the main control cost visible. In the representative run shown here, `CTRL_MM_FEED` and `CTRL_MM_WAIT` dominate the run window, while drain and writeback contribute a second large block of time. Fetch and decode are small by comparison. This is useful later in the performance discussion because it shows that the integration gap is not just "software overhead"; part of it is still control and synchronization work inside the accelerator run window itself.

3.3 Quantizer and Model Preparation

The quantization path exists to turn trained models into artifacts the compiler can actually lower. TinyNPU does not assume that generic floating-point PyTorch modules can be handed directly to the accelerator. The model-preparation code therefore carries explicit quantization metadata, compiler-ready conversion utilities, and calibration steps that make tensor ranges and signedness visible before compilation.

In the current MNIST case study, this pipeline is exercised on a small four-layer model with three 3×3 convolutions followed by global average pooling and a final fully connected layer. The input is a $1 \times 28 \times 28$ image; the learned path is `Conv(1,16)` $\rightarrow$ `Conv(16,16)` $\rightarrow$ `Conv(16,16)` $\rightarrow$ `mean(H,W)` $\rightarrow$ `Linear(16,10)`. On the saved trained run used in this report, the FP32 model reached 77.69% test accuracy, the post-training quantized initialization reached 66.38%, and the QAT-refined model

recovered to 77.56%. This is the concrete role of the quantization subsystem in the stack: it preserves model quality closely enough that the resulting compiler-ready artifact still represents the intended trained model rather than a numerically unrelated toy export.

This part of the stack also includes the current mixed-precision helper. Its purpose is limited and should be stated clearly: it is not a global bit-allocation solver. Instead, it provides a practical one-layer-at-a-time sensitivity workflow that can generate usable mixed-precision configurations, recalibrate them, and pass the result onward to the compiler-ready conversion path. In other words, the flow does not force one model-wide bitwidth; it can keep more sensitive tensors at higher precision while assigning narrower precision to tensors that tolerate it.

The QAT path also became the place where accelerator-specific nonlinear approximations were made explicit rather than being treated as an afterthought. For the integer sigmoid and hard-GELU experiments, QAT was used not merely to recover from generic quantization noise, but to train against the same broad nonlinear contract that the compiler would later freeze into the deployed artifact. In the GELU case, this included learning a per-layer hard-GELU input-domain shift that was later emitted as instruction metadata for the PPU. This is a more realistic role for QAT in TinyNPU than simply toggling fake quantizers: it becomes the stage where approximation-aware activation parameters are learned before the compiler lowers them into a static instruction stream. The integer nonlinear direction itself is motivated by work such as I-LLM and I-BERT, even though the final TinyNPU hard-GELU form is a project-specific hardware approximation rather than a verbatim paper operator [7, 9].

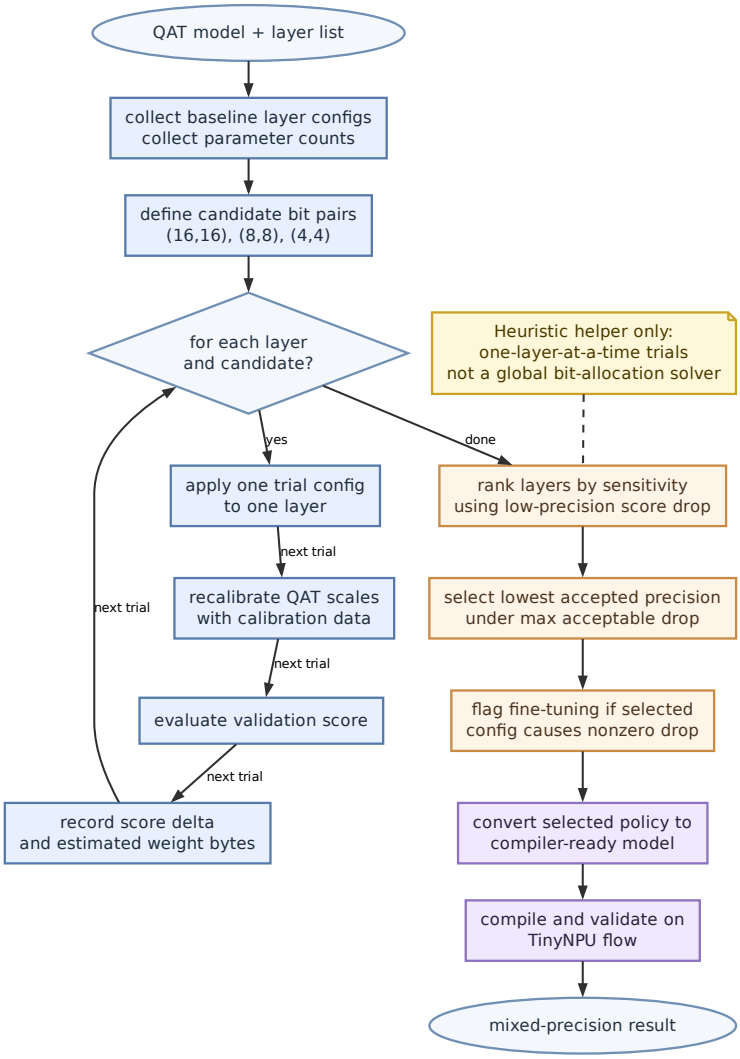

Figure 3.5: Heuristic mixed-precision selection flow used to obtain compiler-ready model variants.

The important role of this subsystem is therefore not to solve quantization research as a whole. It is to provide a disciplined bridge from trained or quantized models into forms that the TinyNPU compiler can lower without losing track of numerical meaning. In the current trained-MNIST path, the compiled host and RTL executions are validated against each other on sampled inputs, so the quantized model-preparation flow is connected all the way through segmented compilation and RTL-backed execution rather than stopping at an offline export boundary.

3.4 Compiler

The compiler is responsible for converting model structure into a mixed execution structure. Rather than generating a monolithic accelerator-only program for the whole

model, the compiler constructs an `ExecutionPlan` whose steps are typed as `NpuSegment`, `HostOp`, or `VerifyTensor`. `NpuSegment` groups accelerator-supported matrix-style work. `HostOp` keeps unsupported or intentionally host-side work separate. `VerifyTensor` allows compiler-owned expected tensors to be checked later at runtime.

This representation is the main reason the stack remains analyzable. Unsupported behavior is not folded into a fake accelerator-only view, and accelerator boundaries are not implicit. That is a deliberate policy: if someone wants to target TinyNPU effectively, the model and compiler flow should reflect the actual hardware boundaries rather than pretending full-model coverage already exists.

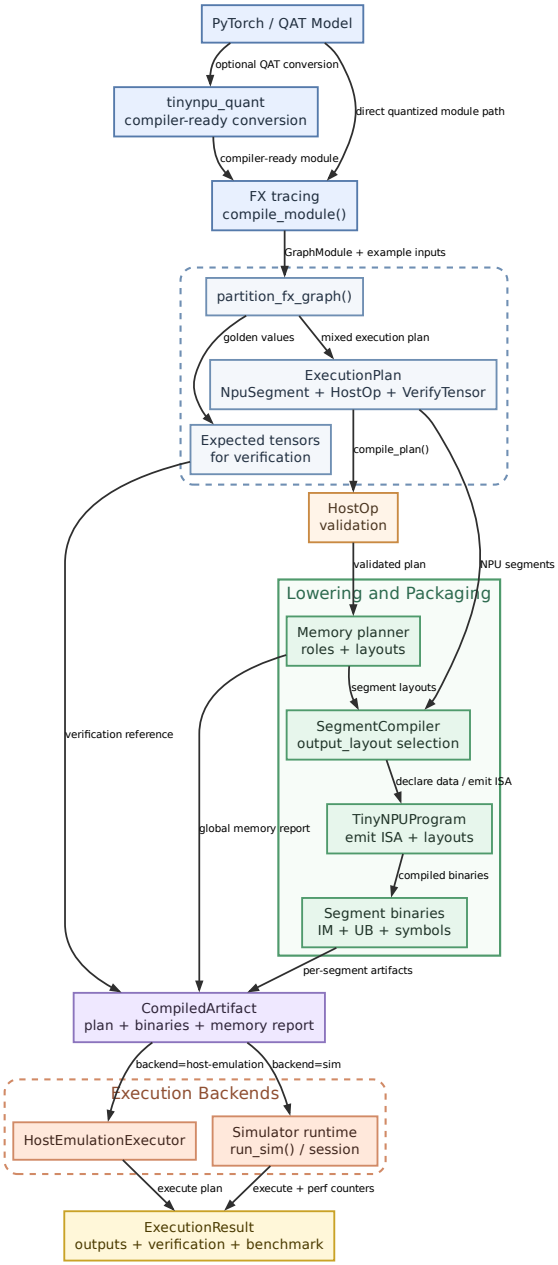

Figure 3.6: Compiler and runtime flow used by the TinyNPU JIT path.

Figure 3.6 summarizes the implemented software flow. A model enters either directly as a supported quantized PyTorch module or through a compiler-ready conversion path built around `tinynpu_quant`. The JIT frontend traces the model with FX, partitions it into a mixed execution plan, validates host operations, and lowers each accelerator segment through the memory planner and `SegmentCompiler`. The resulting `CompiledArtifact` contains the execution plan, per-segment IM/UB binaries, expected tensors for verification, and the memory-layout information required by the runtime.

Packing is part of the compiler contract as well. The backend uses role-specific physical layouts for activation-side tensors, weight-side tensors, output buffers, and bias payloads. These are physical storage forms rather than semantic tensor meanings, but they are essential because legal execution depends on matching memory layout to the real feed directions and writeback format of the NPU. A useful correction came late in the project: internal NPU outputs can no longer be treated as if ordinary C-style writeback were automatically consumable by a following A-side input. The compiler now annotates the required physical output layout explicitly, and the emitted instruction stream carries that selection into the RTL writeback path.

The important compiler policy is not the cycle-by-cycle tile schedule but where accelerator regions begin and end. The partitioner emits a mixed execution plan and forces a segment boundary whenever execution must return to the host, a verification point is requested, or an operation is outside the current NPU contract. As a result, host-side work such as dequantization, softmax, mean, layout restoration, and verification is represented directly as `HostOp` or `VerifyTensor` steps rather than being buried inside an accelerator launch.

Memory planning follows the same policy. The unified buffer is divided into a static zone and a dynamic zone: weights and biases receive globally assigned packed addresses that can be shared across segments, while activations and outputs are placed in a per-segment dynamic region with liveness-based reuse. The planner also treats layout as a real physical requirement. If a tensor must be emitted in an A-compatible layout for an internal chained consumer, that requirement is carried in the memory report rather than being inferred informally. At host-visible boundaries, the current validated flow preserves a host-friendly INT16 representation where possible, especially for boundaries that will be read back or consumed by host operations. The reason is simple: when data returns to the host, the runtime prefers to hand off the highest precision the current datapath can produce instead of requantizing early and making host-side functions such as softmax or verification unnecessarily lossy.

3.5 Runtime and Verification

The runtime executes the compiled artifact as a mixed host-accelerator loop. For each step in the execution plan, it either launches an accelerator segment, executes an explicit host operation, or checks a verification tensor. The key point is that host-side work is never hidden. If a step remains on the host, it remains visible in the plan and in the benchmark accounting.

The runtime also owns repeated-run behavior. It can preserve reusable state such as static unified-buffer contents and preloaded instruction-memory images across runs. This does not remove all setup work, but it makes repeated inference more realistic and exposes a measurable optimization path for reducing avoidable overhead.

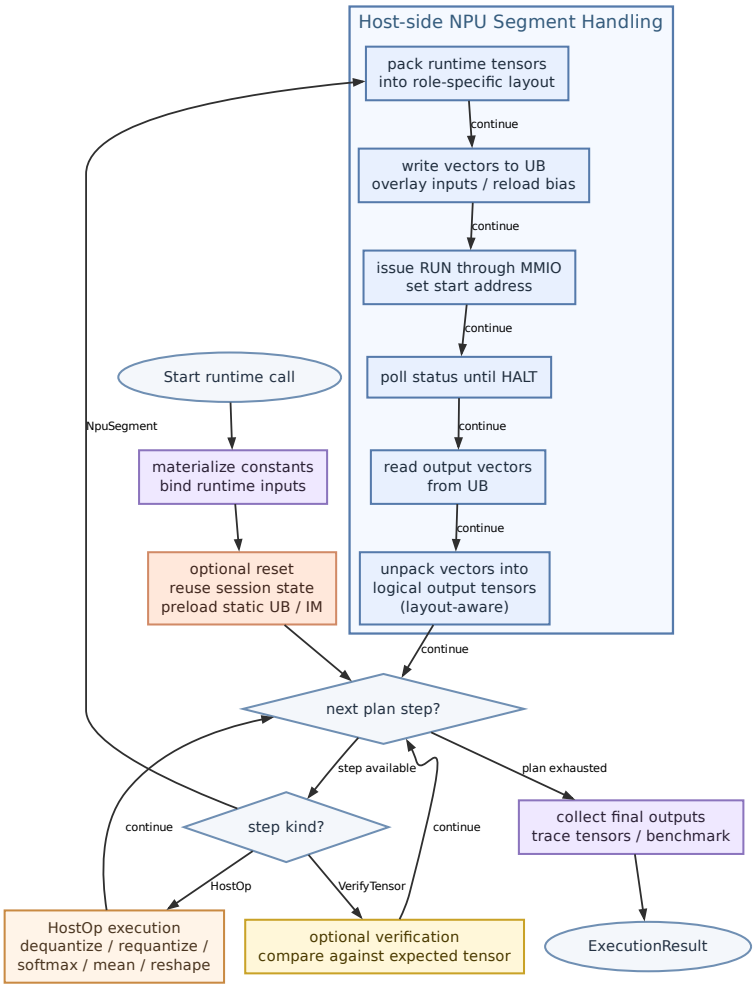

Figure 3.7: Host-side runtime flow for executing a compiled TinyNPU artifact.

Figure 3.7 shows the same loop from the host-side orchestration perspective. For accelerator segments, the runtime first packs logical tensors into the physical layouts expected by the datapath, writes the necessary vectors into the unified buffer, launches execution through MMIO, waits for completion, and then reads and unpacks the resulting vectors back into logical tensors. Host-side operations such as requantization, dequantization, softmax, mean, and shape movement appear explicitly in the same loop rather than being hidden behind the accelerator interface. With the newer output-layout support, the host-side runtime no longer has to repair certain internal layout mismatches itself: when the compiler marks a chained tensor as needing A-format storage, the hardware segment writes it back in that form directly.

Verification is built into this same path. Compiler-owned expected tensors can be

checked during execution, which means correctness is not limited to end-of-run output comparison. Benchmarking is also integrated here: the runtime can record hard RTL cycle counts for accelerator run windows while separately accounting for software-side overhead such as packing, MMIO traffic, and readback.

3.6 Current Scope Boundaries

The current system targets a narrow but concrete contract. The mixed-precision helper produces compiler-ready configurations rather than solving the global bit-allocation problem. The compiler supports a bounded operator set and routes remaining work through host steps. The evaluation focuses on one end-to-end model and one controlled kernel benchmark, which is enough to study the interaction between architecture, compilation, and runtime without overstating workload coverage.

4 Experimental Setup and Design

Evaluation uses three evidence sources. Unit and regression tests verify correctness incrementally across the stack, including compiler transformations, execution-plan behavior, memory planning, and host-side operator semantics. RTL-backed cocotb tests then exercise the hardware/software path directly and expose the control-unit performance counters during execution. A reproducible Yosys flow provides rough logic and DSP estimates for the current top-level design, with and without abstracted memories.

Host emulation and RTL simulation execute the same compiled plan, so the same artifact can be checked both for tensor-level correctness and for hardware-visible cost. The control unit reports both total run-window cycles and per-state cycle and entry counts, which makes it possible to separate time spent in feed, wait, drain, writeback, and related control states.

The empirical results are organized around two workloads. The first is the trained MNIST pipeline used as an end-to-end validation case. The second is a controlled multi-tile GEMM chosen to force tiling on the current array and expose the split between array throughput and integration overhead.

The reported metrics keep those cost categories separate. Accelerator cycles are taken from the RTL run window of the NPU segment itself. Packing, MMIO traffic, tensor movement, and host-only operators are accounted for outside that run window. The CPU baselines used later in the report are analytic reference models for the same logical work, not measurements from a physical comparison platform.

5 Comparative Evaluation and Discussion

This section focuses on two concrete artifacts: the trained MNIST flow as an end-to-end validation case, and the controlled multi-tile GEMM benchmark in Table 5.1. The benchmark metrics are meant to separate core accelerator behavior from host-side cost; the benchmark-cost-model appendix defines the cost model, the four benchmark buckets, and the three reported speedup quantities in detail.

5.1 Kernel-Level Benchmark

Table 5.1: Controlled multi-tile matrix multiplication benchmark results for the 13×17 by 17×19 workload.

Reference model	CPU replaced cycles	Pure acceleration speedup	Integration-adjusted speedup
Unpipelined scalar	51870	140.57×	8.39×
Ideal issue-1	30628	83.00×	5.20×
Five-stage in-order	43472	117.81×	7.09×

For this 13×17 by 17×19 GEMM, the measured accelerator run window was 369 cycles for the NPU segment itself. Read Table 5.1 together with Appendix B: "CPU replaced cycles" estimates how much scalar work the same GEMM would require, "pure acceleration speedup" compares that work against the measured NPU run window, and "integration-adjusted speedup" adds the surrounding packing, MMIO, and runtime cost. The large gap between the last two columns is therefore the main result of the table: the core compute block is strong, but software and interface overhead still consume a large part of end-to-end execution.

The main takeaway from Table 5.1 is therefore not the exact headline speedup value, but the split between inner compute strength and delivered system speedup. TinyNPU already replaces a substantial amount of scalar MAC work in a short RTL run window, yet that advantage is diluted once the cost of using the accelerator is included. For an examiner, the correct conclusion is that the arithmetic core is not the main weakness of the current stack; the limiting factor is the amount of movement and orchestration wrapped around each accelerator region.

To connect this kernel benchmark to related low-bit GEMM work, Table 5.2 reports the TinyNPU quantities that overlap with the comparison boundary used for Mix-GEMM [10]. The shared boundary is precision scaling in tiled GEMM kernels, not end-to-end software-stack latency. TinyNPU therefore reports RTL-measured GEMM throughput and a region-level speedup under one fixed TinyNPU host model, while Mix-GEMM reports performance and speedups against OpenBLAS on a SiFive U740. The absolute speedup values are not interchangeable, but the precision-scaling trend is.

Table 5.2: Kernel-level TinyNPU GEMM scaling used for the Mix-GEMM comparison boundary. Region-level speedup uses the ideal issue-1 reference model from Appendix B.

Workload	INT16 MAC/cycle	INT8 MAC/cycle	INT4 MAC/cycle	INT8/16 gain	INT4/8 gain	INT4 speedup
64^3	41.24	60.84	79.80	1.48×	1.31×	32.19×
$96 \times 64 \times 96$	41.30	60.98	80.04	1.48×	1.31×	37.18×
128^3	50.22	82.65	122.06	1.65×	1.48×	62.99×

Table 5.2 also makes one important limitation visible. If throughput scaled only with packed lanes, INT8 would deliver 2× the logical MAC rate of INT16, and INT4 would deliver another 2× over INT8. In the present GEMM set, TinyNPU achieves 1.48–1.65× for INT8/INT16 (74–83% of ideal) and 1.31–1.48× for INT4/INT8 (66–74% of ideal). Mix-GEMM reports closer-to-ideal scaling in the same broad sense; for example, their a5-w5 configuration is reported as a 60% improvement over a8-w8 (1.6×), which matches the ideal 8/5 scaling in that step [10]. The gap in TinyNPU scaling is expected for small-to-medium tiles because fill, drain, and control costs remain roughly fixed while the number of logical MACs per compute cycle rises with narrower packing.

The same table can also be read against the array's theoretical peak. For the current 8×8 array, the peak issue rates are 64 INT16 MACs/cycle, 128 INT8 MACs/cycle, and 256 INT4 MACs/cycle. On the largest reported kernel, 128^3, TinyNPU reaches 50.22, 82.65, and 122.06 MACs/cycle, which corresponds to 78.5%, 64.6%, and 47.7% of those respective peaks. This makes the scaling story clearer than gain ratios alone: larger kernels improve utilization by amortizing fill and drain overhead, but utilization still drops as precision narrows because the fixed control and wavefront costs consume a larger fraction of the run while the logical MAC count per cycle rises.

5.2 Repeated-Run Execution-Plan Case Study

Repeated-run persistence is also justified by measured behavior. In the trained MNIST flow, keeping stable state resident across runs reduces unified-buffer write traffic from 17608 to 14920 words per inference, which corresponds to a 15.3% reduction.

That 15.3% reduction should not be read as a dramatic model-level speedup by itself. Its importance is narrower and more practical: it shows that a measurable fraction of repeated-run cost still comes from avoidable state movement rather than from arithmetic. In other words, the runtime is not yet "done" once the math works; keeping stable data resident across inferences already removes a non-trivial amount of protocol traffic.

The clearest way to interpret the current performance story is as a two-case comparison: one workload that fits the accelerator's native contract well, and one workload that repeatedly falls back to host-side layout helpers. To make that comparison fairer than the earlier mixed-precision / global-pooling example, both paths in Table 5.3 use the

same binary `is_zero` task on resized 8×8 MNIST and keep all layer boundaries at INT16. The benchmark artifact used for persistence measurement also keeps the final output quantized, so the listed execution plans are the literal persisted plans used in the RTL runs.

Table 5.3: Case-study comparison using literal forward-style model definitions, literal execution-plan structure, and steady-state delivered speedup.

Conv-heavy MNIST feature model	**One-segment MLP feature model**
Forward definition	

`x = F.relu(conv1(x))`
`x = F.relu(conv2(x))`
`x = F.relu(conv3(x))`
`x = torch.sigmoid(conv4(x))`
	`x = F.relu(fc1(x))`
	`x = F.relu(fc2(x))`
	`x = F.gelu(fc3(x))`
	`x = torch.sigmoid(fc4(x))`

Execution plan	

`HostOp(quantize)`
`HostOp(im2col)`
`NpuSegment(conv1 + relu)`
`HostOp(layout_restore)`
`HostOp(im2col)`
`NpuSegment(conv2 + relu)`
`HostOp(layout_restore)`
`HostOp(im2col)`
`NpuSegment(conv3 + relu)`
`HostOp(layout_restore)`
`HostOp(im2col)`
`NpuSegment(conv4 + sigmoid)`
`HostOp(layout_restore)`
	`HostOp(quantize)`
	`NpuSegment(fc1 + relu,`
	`  fc2 + relu,`
	`  fc3 + h_gelu,`
	`  fc4 + sigmoid)`

$S_{\mathbf{end}}$	
$6.43\times$	$15.49\times$

This comparison explains why the conv-heavy path "comes back to bite" the delivered speedup even though the arithmetic core is strong. The convolution model is not slow because its MAC regions are intrinsically weak; it is slow because each convolution crosses the host/NPU boundary twice for layout reasons. The host must materialize sliding-window matrices through `im2col`, the NPU processes that matrix multiplication region, and then the host must rebuild feature-map layout through `layout_restore` so the next convolution can start from a normal tensor again. Those helpers are real remaining work, and under the stricter benchmark split they count against the delivered speedup. The one-segment MLP avoids exactly those penalties because it never leaves the linear-native contract of the array.

A repeated-run experiment on the one-segment MLP shows the clean best-case path. After fixing an internal C-to-A layout mismatch by adding output-layout-aware write-back, the compiled host and RTL paths matched on a balanced 30-sample sweep with 15 positive and 15 negative examples. Under the ideal issue-1 reference model, the reused accelerator run window stayed at 2551 cycles per inference, while NPU overhead dropped from 2919 cycles on the first run to a stable 2729 cycles on runs 2–30.

Only the initial `quantize` helper remained in the persisted host plan, so the unified host-remaining bucket was also only 448 cycles per inference in both the cold and warm cases. This corresponds to a pure accelerator speedup of 34.78× in both cases, an integration-adjusted speedup improving from 16.22× on the first run to 16.80× in steady state, and a stricter end-to-end analytical speedup improving from 14.99× to 15.49× once the remaining host helpers are also counted.

The conv-heavy feature model provides the complementary result under the same INT16 / `is_zero` setup. In that case the first run measured $C_{\text{cpu}} = 391{,}466$, $C_{\text{npu}} = 1{,}778$, $C_{\text{ovh}} = 35{,}795$, and $C_{\text{host}} = 23{,}644$ cycles. Runs 2–30 then stayed at the same warm averages for compute and remaining host work, while NPU overhead dropped slightly to 35,491 cycles. The pure accelerator speedup is therefore still 220.17×, but the integration-adjusted speedup is only 10.50×, and the stricter end-to-end analytical speedup falls further to 6.43×.

That pair of case studies is the most useful current performance result in the report. In the one-segment MLP, the host boundary is almost absent and the delivered speedup stays close to the integration-adjusted figure. In the conv-heavy path, by contrast, the pure accelerator number is misleadingly optimistic if taken alone. The host still executes `im2col`, `layout_restore`, and the surrounding orchestration on every convolution boundary, so the delivered speedup is less than half the one-segment case even though the raw compute replacement number is much larger. This is why reducing host-visible layout work is now a more important next step than trying to make the core MAC datapath itself more aggressive.

5.3 Activation Study

The activation study reduced to one clear result. Hard-GELU is functionally implementable in TinyNPU, but `W8A8` hard-GELU is too lossy on the trained MNIST flow. The FP32 reference reached 71.54%, while the corresponding `W8A8` PTQ model fell to 12.14% and QAT recovered only to 54.44%. Keeping the hard-GELU boundary at INT16 instead changed the outcome substantially: PTQ improved to 35.53% and QAT recovered to 69.98%, close to the FP32 reference. Hard-GELU is only viable when the post-activation boundary retains sufficient precision. The full INT16 variant then exposed the next limit by exceeding the current unified-buffer budget, so the practical next step is selective INT16 retention around sensitive GELU boundaries rather than blanket INT16 everywhere.

5.4 Bottlenecks and Interpretation

Taken together, the current results point to a fairly clear bottleneck ranking. Inside the accelerator run window, Figure 3.4 shows that feed and wait dominate, so control and wavefront timing still matter even before software overhead is counted. At the system level, however, the larger limitation is the cost of launching and servicing relatively small accelerator regions: packing, MMIO traffic, readback, and host-visible

boundaries dilute the raw core speedup more than fetch/decode overhead does. The next optimization most likely to improve integration-adjusted speedup is therefore not a more aggressive PE datapath, but reducing avoidable boundary cost by fusing larger accelerator regions, preserving reusable state across runs, and cutting unnecessary pack/unpack and readback traffic.

A rough synthesis snapshot is included only for scale. The current Yosys flow estimates 25117 logic cells when the major memories are abstracted and 33644 logic cells when the current memory implementation is kept, with 227 DSP48 blocks in both cases. These figures should now be treated as *stale synthesis proxies* rather than current area numbers, because the RTL and software stack have continued to evolve after that snapshot. They are not final silicon area, but they are still sufficient to place the design in a non-trivial hardware range.

6 Conclusion and Future Work

TinyNPU is a working mixed-precision accelerator platform with a usable hardware architecture, a PyTorch-oriented model-preparation flow, a segmented compiler/runtime, explicit host-boundary handling, repeated-inference support, verification infrastructure, and benchmark tooling that separates compute from overhead.

The results show a clear split between compute capability and integration cost. The systolic core delivers strong throughput on tiled GEMM, but practical execution still pays a noticeable cost for segmentation, state movement, packing, and host-visible boundaries. The mixed-precision path is practical for preparing compiler-ready models, and the current evaluation focuses on one end-to-end model and one controlled kernel benchmark.

The immediate priorities are to bring more models onto the stack, reduce avoidable runtime and interface overhead, increase the size of useful accelerator regions, and use measured workloads to justify changes in unified-buffer and instruction-memory sizing. The hard-GELU study sharpened that agenda: richer nonlinearities are feasible in the current PPU, but only if the precision-memory tradeoff is handled explicitly. W8A8 hard-GELU was too lossy on the trained MNIST flow, while an INT16-preserving variant recovered nearly all of the FP32 accuracy but exceeded the present unified-buffer budget. That makes selective precision retention around sensitive activation boundaries a more useful next target than simply adding more nonlinear modes. On the hardware side, synthesis and timing characterization should be refreshed, because the logic-cell figures quoted earlier are now stale proxy numbers rather than up-to-date implementation results. On the software side, compiler work should continue to target the boundary cases and data-movement costs that show up in measured workloads.

A Implemented Activation Algorithms

This appendix records the activation functions as they were actually implemented in the TinyNPU software golden path and mirrored in the RTL primitives. These algorithms therefore describe the deployed integer contract rather than an idealized floating-point reference.

Require: Integer input x_{in}, integer parameters $m_I > 0$, $k_I \geq 0$
Ensure: Integer approximation of an exponential-like decay

$m_F \leftarrow m_I + (m_I \gg 1) - (m_I \gg 4)$
$s_I \leftarrow \left(2^{k_I} + \lfloor m_F/2 \rfloor\right) / m_F$
$t \leftarrow -s_I$
$q \leftarrow \lfloor x_{in}/t \rfloor$
$r \leftarrow x_{in} - q \cdot t$
$u \leftarrow (r \gg 1) - t$
$y \leftarrow u \gg q$
return $\max(0, y)$

Algorithm A.1: TinyNPU DI-Exp

Require: Integer input x_{in}, integer parameters m_I, k_I, output precision parameter p_{out}, optional smoothing factor α_{smooth}
Ensure: Integer sigmoid approximation in a signed p_{out}-bit probability domain

$x_s \leftarrow \lfloor x_{in}/\alpha_{smooth} \rfloor$
$e_0 \leftarrow \text{DI-Exp}(0, m_I, k_I)$
if $x_s \geq 0$ **then**
    $e \leftarrow \text{DI-Exp}(-x_s, m_I, k_I)$
    $n \leftarrow e_0$
else
    $e \leftarrow \text{DI-Exp}(x_s, m_I, k_I)$
    $n \leftarrow e$
end if
$d \leftarrow e_0 + e$
$q_{max} \leftarrow 2^{p_{out}-1} - 1$
return $(n \cdot q_{max} + \lfloor d/2 \rfloor) / d$

Algorithm A.2: TinyNPU DI-Sigmoid

Require: Signed integer input x_{in}, activation-domain shift k_x, slope parameters (n_s, k_s) approximating $1.702 \approx n_s/2^{k_s}$
Ensure: Signed integer hard-GELU result in the same activation domain

$S \leftarrow 2^{k_x}$
$t_3 \leftarrow 3S$
$t_6 \leftarrow 6S$
$s \leftarrow \text{RoundShiftSigned}(x_{in} \cdot n_s, k_s)$
$g \leftarrow \min(\max(s + t_3, 0), t_6)$
$y \leftarrow \text{RoundDivSigned}(x_{in} \cdot g, t_6)$
return y

Algorithm A.3: TinyNPU Hard-GELU

B Benchmark Cost Model

The benchmark path used in this project combines measured RTL cycle counts with an analytic reference-CPU model. The goal is not to claim a definitive comparison against a specific commercial processor. Instead, it is to make the accelerator results interpretable by separating four different questions:

1. How much logical work would a scalar CPU have performed for the same operation?
2. How many cycles did the accelerator actually spend in its measured run window?
3. How much extra cost was introduced by packing, MMIO traffic, and host-side orchestration?
4. How much work still remained on the host because the compiled plan retained helper operations outside accelerator segments?

To support that split, benchmark entries are grouped into four buckets:

1. **CPU-replaced work** ($C_{\mathbf{cpu}}$)**:** logical work that the CPU would have performed instead of an accelerator segment. For matrix multiplication, this includes scalar reads, multiply-accumulate work, loop-control branches, optional bias addition, requantization, and output writes.
2. **Accelerator compute** ($C_{\mathbf{npu}}$)**:** measured RTL cycle counts collected during the accelerator run window with performance monitoring enabled.
3. **Accelerator overhead** ($C_{\mathbf{ovh}}$)**:** software-visible and protocol-visible work required to use the accelerator, such as packing, interface writes, interface reads, instruction loading, and readback.
4. **Remaining host work** ($C_{\mathbf{host}}$)**:** all helper work outside accelerator segments that still executes on the host. This includes `quantize`, `dequantize`, `requantize`, `softmax`, `mean`, `reshape`, `transpose`, `layout_restore`, `im2col`, and alias-style movement helpers.

Primitive work is first counted in an abstract instruction-class form. The counted fields include reads, writes, adds, multiplies, branches, shifts, clamps, bitwise operations, divides, nonlinear helper calls, interface writes, and interface reads.

An analytic CPU model is then applied to those counts:

$$\texttt{cycles} = \sum_i \texttt{count}_i \times \texttt{cost}_i$$

The three reference models currently used in the project are shown in Table B.1. These are implemented directly in the benchmark code and reuse the same counted trace; only the CPI assumptions change.

Table B.1: Reference CPU cost models used in the TinyNPU benchmark flow.

Event class	**Unpipelined scalar**	**Ideal issue-1**	**Five-stage in-order**
Reads / writes	2 / 2	1 / 1	1 / 1
Adds / shifts / clamps / bitops	1	1	1
Integer multiply	4	1	3
Branch	1	1	2
Divide	16	1	16
Nonlinear helper	8	1	8
Interface read / write	1 / 1	1 / 1	1 / 1

Using those symbols, the reported speedups are:

$$S_{\text{pure}} = \frac{C_{\text{cpu}}}{C_{\text{npu}}}$$
$$S_{\text{int}} = \frac{C_{\text{cpu}}}{C_{\text{npu}} + C_{\text{ovh}}}$$
$$S_{\text{end}} = \frac{C_{\text{cpu}}}{C_{\text{npu}} + C_{\text{ovh}} + C_{\text{host}}}$$

This definition is why the integration-adjusted and end-to-end analytical speedups are always lower than the pure accelerator speedup. S_{int} includes the cost of using the accelerator, not only the measured run window inside the hardware. S_{end} then tightens that denominator further by adding all remaining host-side helper work that the compiled plan still executes outside accelerator segments. That is an intentional design choice in the methodology: the project is trying to understand the full cost of practical use, not only the inner datapath in isolation.

References

[1] Georgios Armeniakos, Francesco Maria Ottavi, Angelo Garofalo, and Luca Benini. Mixed-precision support for RISC-V soft-SIMD extensions, 2024.

[2] Xander Chin, Kenny Guo, Evan Lin, and Surya Sure. Tiny-tpu: the why and how. https://www.tinytpu.com/, August 2025. Accessed March 20, 2026.

[3] Angelo Garofalo, Francesco Conti, Giuseppe Tagliavini, Davide Rossi, and Luca Benini. XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based iot end nodes. *IEEE Transactions on Emerging Topics in Computing*, 10(4):1849–1863, 2021.

[4] Hasan Genc, Ameer Haj-Ali, Vivek Ittycheriah, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration. *Proceedings of the 58th Annual Design Automation Conference (DAC)*, pages 769–774, 2021.

[5] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):12:1–12:25, 2008.

[6] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2016.

[7] Xing Hu, Yuan Cheng, Dawei Yang, Zhihang Yuan, Jiangyong Yu, Chen Xu, and Sifan Zhou. I-llm: Efficient integer-only inference for fully-quantized low-bit large language models, 2024.

[8] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.

[9] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. I-bert: Integer-only bert quantization. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 5506–5518, 2021.

[10] Enrico Reggiani, Alessandro Pappalardo, Max Doblas, Miquel Moreto, Mauro Olivieri, Osman Sabri Unsal, and Adrian Cristal. Mix-GEMM: An efficient hw-sw architecture for mixed-precision quantized deep neural networks inference on edge devices. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.

[11] Bichen Wu, Yanghan Wang, Peizhao Zhang, Yonglong Tian, Peter Vajda, and Kurt Keutzer. Mixed precision quantization of convnets via differentiable neural architecture search, 2018.

[12] Guangxuan Xiao, Ji Lin, Meryem Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 38087–38099, 2023.